

KOBIE LAUNCHPAD AI HACKATHON

Solution Presentation

InfoVac

Track D : Autonomous Competitive Intelligence Agent

Team Members

Shashank U - Backend & Data Pipeline

Hrishikesh R Prasad - LLM Integration & Agent Architecture

B. M. S. College of Engineering

Problem & Our Approach



THE CORE CHALLENGE

What is the real challenge — not just the surface task?

Loyalty program data** is scattered, contradictory, and goes stale fast. LLMs make this worse by confidently inventing specifics instead of admitting uncertainty. The real challenge is resisting the guess. An LLM will confidently invent an answer if you let it, so the system is built to **reward honesty over confidence**, since **wrong answers cost 3x more than "unknown."



OUR KEY INSIGHT

What is the single core idea behind your solution?

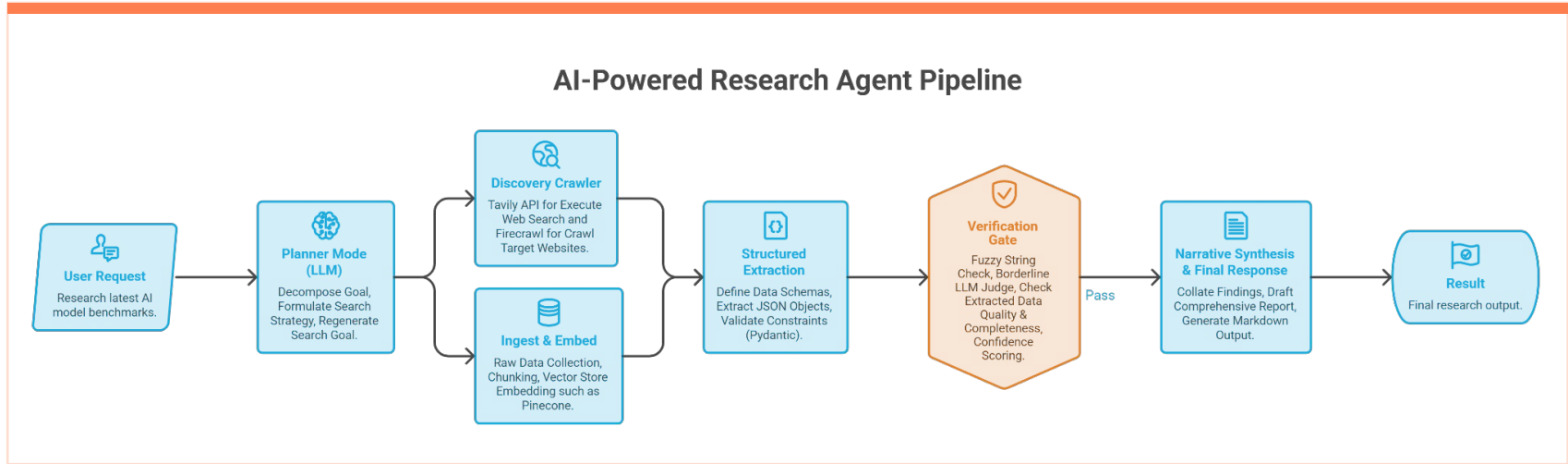
*LLMs are confident even when wrong. So instead of trusting generation, the system verifies it. A **deterministic gate (Rapid Fuzz)** checks each claim against source text, nullifying anything unmatched instead of trusting it. **Misattributed citations get corrected automatically by scanning all crawled sources.** Only genuine paraphrase ambiguity goes to an LLM judge, keeping the fast, deterministic path as default authority.*

The payoff: every claim in the output is traceable to a real document.

DATA QUALITY: What issues did you discover in the data during EDA? How did they influence your approach?

*Scraped text arrived with **encoding corruption**, silently breaking matches unless repaired first. Naive scraping also stripped the HTML tables where tier and earn data live, so **tables get reconstructed before extraction.** Sources vary in **reliability and freshness** (T&Cs weighted higher than forums, older data decays), and outright contradictions get capped in confidence and flagged for human review instead of silently averaged.*

Architecture & Technical Design



KEY TECH CHOICES

LangGraph – Stateful workflow orchestration.

Instructor + Pydantic – Schema-validated LLM outputs.

DESIGN DECISIONS

1. Weakest-link composite quoting + zero-token regex classifier
2. Append-only field history + strict typed state

ERROR HANDLING

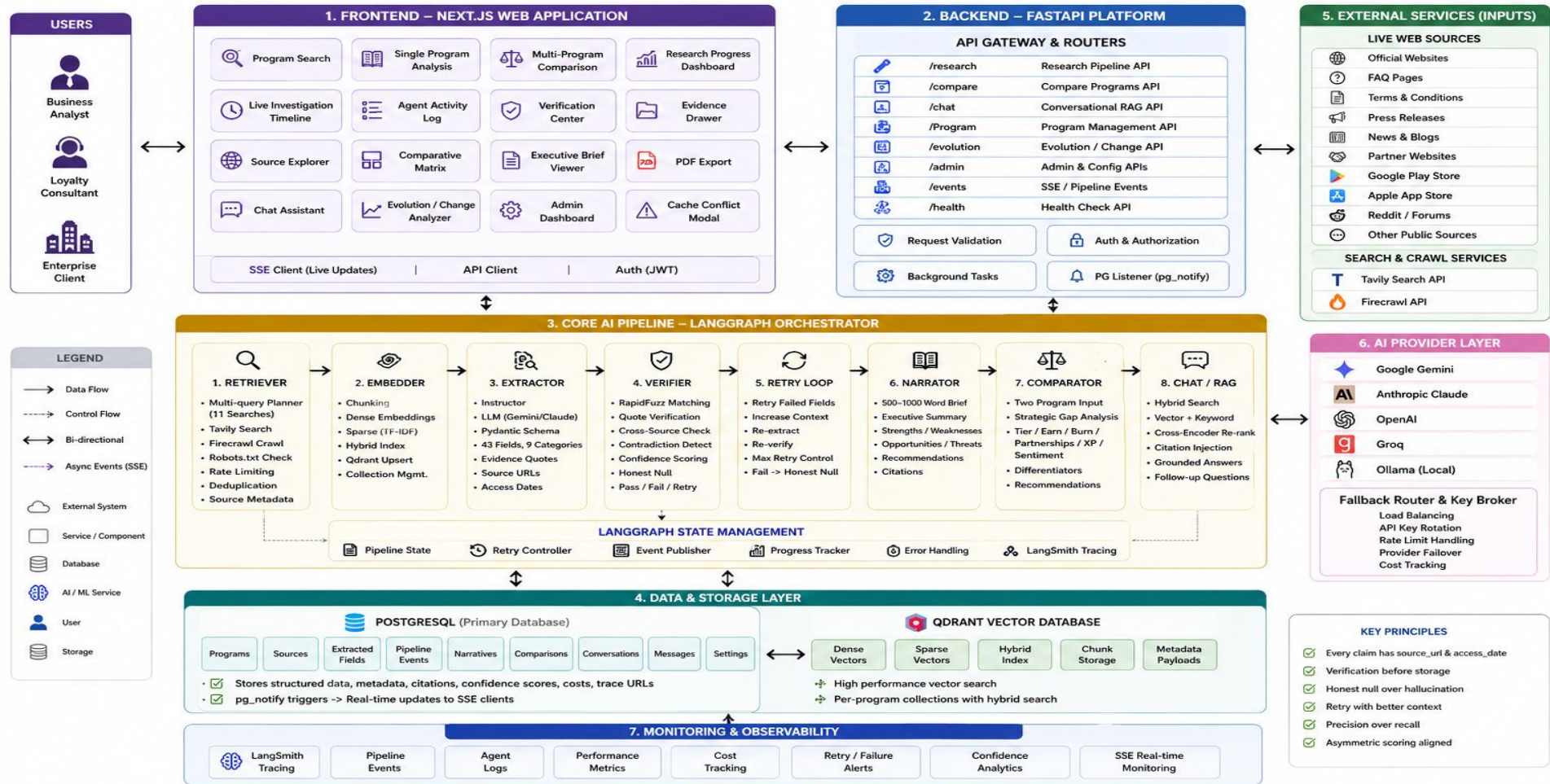
Hallucination-resistant: Prefers abstaining over guessing
Failure-resistant – Continues operating despite failures

INFOVAC – AUTONOMOUS COMPETITIVE INTELLIGENCE PLATFORM

HIGH LEVEL SYSTEM ARCHITECTURE

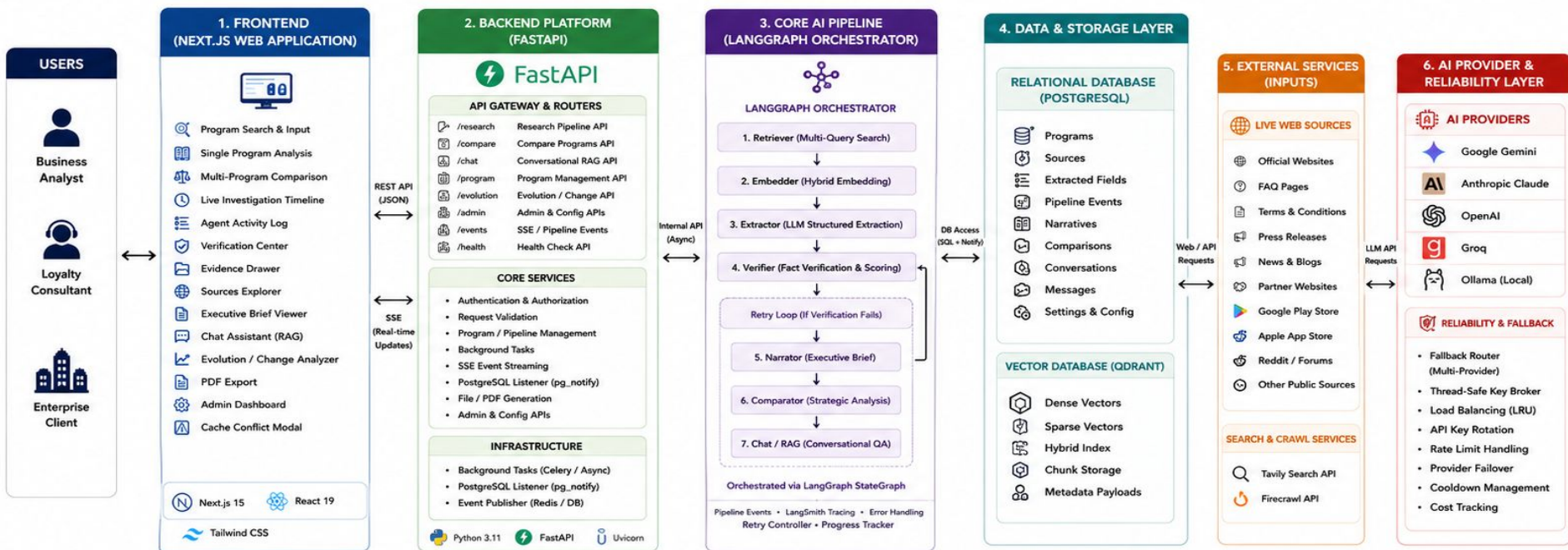
MISSION

Research any loyalty program from the live web, extract 35+ structured fields, verify every claim with sources, generate analyst-grade briefs, compare programs and answer follow-up questions – with zero hallucinations.

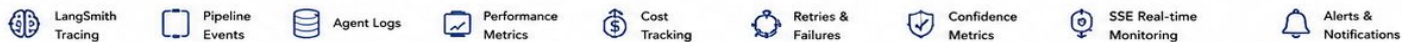


INFOVAC – AUTONOMOUS COMPETITIVE INTELLIGENCE PLATFORM

LEVEL 1: HIGH LEVEL SYSTEM ARCHITECTURE



7. MONITORING & OBSERVABILITY (CROSS-CUTTING)



LEGEND



KEY PRINCIPLES

- ✓ Every claim has source_url & access_date
- ✓ Verification before storage
- ✓ Honest null over hallucination
- ✓ Retry with better context
- ✓ Precision over recall
- ✓ Asymmetric scoring aligned
- ✓ Real-time transparency
- ✓ High reliability & observability

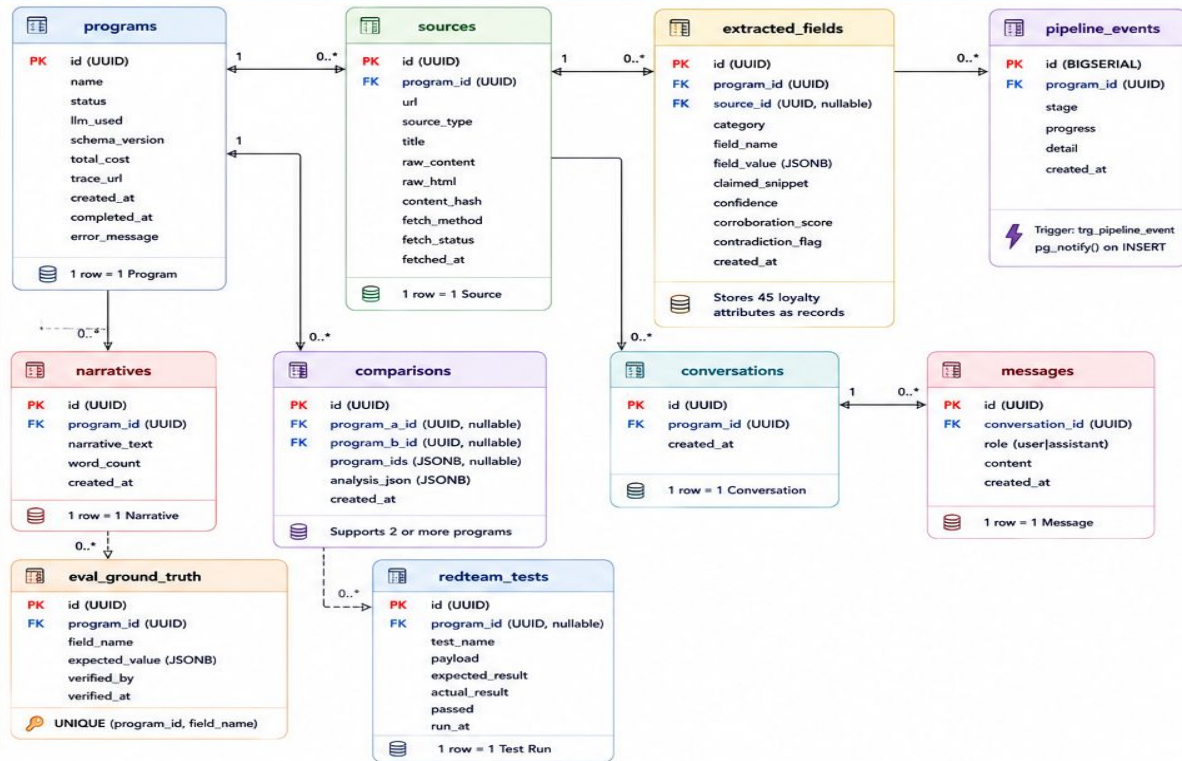


INFOVAC – POSTGRESQL DATABASE ARCHITECTURE

Relational Schema, Relationships & Interactions

PostgreSQL
Primary Database

DATABASE SCHEMA (ER DIAGRAM)



LEGEND

- PK Primary Key
- FK Foreign Key
- 1 One
- 0..* Zero or Many
- Mandatory
- - - Optional / Nullable

DESIGN PRINCIPLES

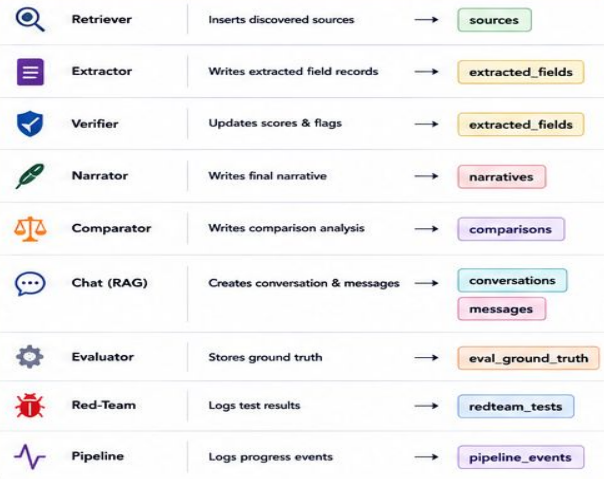
- ✓ Append-only history for extracted fields (no UNIQUE on program_id + field_name).
- ✓ ON DELETE CASCADE used for all dependent relationships.
- ✓ JSONB used for semi-structured data (field_value, analysis_json, program_ids, etc.).
- ✓ UUID primary keys for global uniqueness and scalability.
- ✓ Pipeline events emitted via PostgreSQL trigger + pg_notify for real-time updates.
- ✓ Supports multi-program comparison (2 or more).



LOYALTY EXTRACTION SCHEMA

The system extracts 45 loyalty attributes across 5 categories. These attributes are stored as rows in extracted_fields using: category, field_name, field_value.

DATABASE INTERACTIONS (WRITE / READ FLOW)



REAL-TIME EVENT FLOW (SSE)



TABLE SUMMARY

Table	Purpose	Key Relationships
programs	Main loyalty program records	Parent of most tables
sources	Raw sources discovered	Belongs to programs
extracted_fields	45 loyalty attributes (append-only)	Belongs to programs, sources (optional)
pipeline_events	Pipeline progress tracking (real-time)	Belongs to programs (triggered)
narratives	Generated narrative output	Belongs to programs
comparisons	Comparative analysis (2+ programs)	References programs (nullable)
conversations	Chat sessions per program	Belongs to programs
messages	Chat messages in conversation	Belongs to conversations
eval_ground_truth	Verified ground truth for evaluation	Belongs to programs
redteam_tests	Red-team evaluation results	Belongs to programs (optional)

NOTES: All timestamps are stored in UTC (TIMESTAMPZ). • JSONB used for flexible metadata storage. • Soft deletes via status flags where required. • Backups & snapshots managed at DB level.



1. INGESTION PIPELINE (WRITE PATH)



1. Retriever

Fetch web pages
Save raw content to
PostgreSQL (sources)



2. Embedder Node

Reads Sources from
PostgreSQL



3. Chunking

RecursiveCharacterTextSplitter
• Chunk Size: 600 chars
• Overlap: 100 chars
• Separators: ["\\n\\n", "\\n", ".", ",", ""]
• No parent-child relationship
• Linked via source_id in payload



4a. Dense Embedding

Google API
models/gemini-embedding-2
Dimension: 3072



4b. Sparse Embedding

Local TF-IDF
TfidfVectorizer
(stop_words='english')



5. Upsert to Qdrant (Batch = 100)

Point ID: uuid.uuid4().hex
Timeout: 45.0s
On error: log warning, emit pipeline event,
continue execution (non-blocking)

2. QDRANT COLLECTION (sources)



Qdrant Cloud

Collection: sources (Single Collection)

Dense Vector ("")

- Dimension: 3072
- Distance: Cosine
- Vector Name: "" (unnamed)
- HNSW: Default
- Quantization: None

Sparse Vector ("sparse-text")

- On-disk Sparse Index: Enabled
- Sparse Vector Name: "sparse-text"

Point Structure

Point ID
uuid.uuid4().hex

Dense Vector
""

+ Sparse Vector
"sparse-text"

- Payload (JSON)

Payload Schema

program_id	(string, UUID)
source_id	(string, UUID)
source_url	(string)
source_type	(string) // homepage faq tnc app_review press news forum
text	(string) // raw chunk text

Indexes & Configuration

- HNSW: Default (Qdrant Cloud defaults)
- Quantization: None
- On-disk Sparse Index: Enabled
- Sharding: Managed by Qdrant Cloud

3. RETRIEVAL / CHAT RAG (READ PATH)



1. User Query

User sends question to
API /api/programs/{id}/chat



2. Build TF-IDF Space

Fetch all Program Sources
from PostgreSQL
Fit TfidfVectorizer on all
chunks (program scope)



3. Create Query Vectors

- Dense: Gemini Embedding-2 (3072-d)
- Sparse: TF-IDF vector for query



4. Hybrid Search (RRF)

Qdrant Hybrid (Dense + Sparse)
limit = 20 candidates



5. Rerank Candidates

CrossEncoder
BAAI/bge-reranker-base
Rerank Top 20 → Top 3



6. Build Context

- Top 3 chunks from Qdrant
- Verified facts from PostgreSQL
- Narrative summary from PostgreSQL



7. Generate Answer (LLM)

Gemini / Groq API
Generate final answer



8. Save Chat Exchange

Store user & assistant messages
in PostgreSQL (messages table)

Filters
(Mandatory)

- program_id ==
target_program_id

Optional Filter

- source_type ==
requested_source_type

4. COMPONENT INTERACTION WITH QDRANT

Component	Reads from Qdrant	Writes to Qdrant	Details
Retriever (backend/retriever.py)	✗	✗	Saves pages to PostgreSQL only
Embedder Node (orchestrator/nodes.py)	✗	✓	Chunks, embeds and upserts points
Extractor (backend/extractor.py)	✗	✗	Reads content from PostgreSQL
Verifier (backend/verifier.py, gate.py)	✗	✗	Fuzzy match citations using Postgres tests
Narrator (backend/narrator.py)	✗	✗	Uses Postgres data only
Comparator (backend/comparator.py)	✗	✗	Uses Postgres data only
Chat (backend/chat.py)	✓	✗	Hybrid search & reranking from Qdrant

✓ Uses ✗ Does Not Use

5. EMBEDDINGS & CHUNKING DETAILS



Dense Embedding

- Model: models/gemini-embedding-2
- API: Google Generative AI
- Dimension: 3072
- Used for document chunks and user query



Sparse Embedding (TF-IDF)

- Library: scikit-learn TfidfVectorizer
- stop_words='english'
- Fitted on all chunks of the target program
- Used for document chunks and user query

6. ERROR HANDLING (UPSERT)



Upsert Batch (100 points)



45.0 sec Timeout



Timeout / Exception



Log Warning + Emit Pipeline Event
(to PostgreSQL pipeline_events)



Continue Pipeline Execution
(Non-Blocking)

7. END-TO-END RETRIEVAL FLOW (SUMMARY)



Key Parameters

- Collection: sources
- Dense Dimension: 3072
- Dense Distance: Cosine
- Sparse Vector: sparse-text
- Hybrid Method: Reciprocal Rank Fusion (RRF)
- Top-K (Qdrant): 20
- Rerank Model: BAAI/bge-reranker-base
- Rerank Output: Top 3
- Chunk Size: 600 chars
- Chunk Overlap: 100 chars
- Batch Upsert Size: 100

Design Notes

- Single collection for all program data
- Filtered by program_id (mandatory)
- Optional filter: source_type
- Vectors are not returned in search
- Payload only (text, source_url, etc.)

Results & Key Outputs



Schema & Verification Coverage

The schema covers 45 fields across 8 categories, exceeding the spec's requirement of 35+ fields across 8. Every single field carries its own source URL, timestamp, claimed snippet, gate-pass status, match score, and exact citation offsets, so any value in the system can be traced back to precisely where it came from.



Validation & Telemetry

Our confidence formula is validated against theoretical estimates, and the entire gate, verifier, and RAG pipeline, including hybrid search, reranking, and LLM-judge routing, is covered by automated tests. We target 0.70 Faithfulness and 0.60 on other RAGAS metrics, with a 0.50 floor enforced in code, and every run's cost is tracked to the exact dollar.

Output : A structured PDF with detailed comparisons with all sources cited

Access here : [Google Drive](#)

Business Impact & Activation



ACTIVATION STRATEGY

Program redesign (Retail) → Channel:

Comparison dashboard • **Message:** Tier and benefits benchmark against competitors • **Offer:** Data-driven redesign opportunities • **Timing:** During strategy workshops.

Ongoing monitoring (Hospitality) → Channel:

Program Evolution dashboard + alerts • **Message:** Competitor program changes detected • **Offer:** Early response recommendations • **Timing:** Continuously / after each refresh.



SO WHAT? — THE BUSINESS CASE

Replaces manual research: Converts hours of searching scattered loyalty program information into a single, continuously updatable audit.

Evidence-backed decisions: Every claim is source-cited, so marketing, legal, and compliance teams can independently verify information before using it.

Lower strategic risk: Missing or conflicting information is flagged instead of fabricated, reducing the chance of making business decisions based on incorrect data.

Precision-over-Recall: The system intentionally prefers honest nulls over hallucinations. A missed fact is inexpensive to verify manually; a fabricated fact risks client trust and incurs a much larger scoring penalty.

Live Demo Highlights

Show, don't tell. Walk the judges through a live run or key screenshots.

Live website URL : infovac-kobie.vercel.app/
Github : <https://github.com/Hrishikesh-Prasad-R/info-vac>
Report : [view here](#)

To start locally : `./setup.bat && ./start.bat` (Windows only)

Runs for programs already tested are automatically stored in database and reused instead of a fresh crawl

Learnings, Limitations & Next Steps



WHAT WE LEARNED

- *Verification, not extraction, was the real challenge, an independent check that can't be fooled by re-asking the same model is where the real engineering lives. Led us to RapidFuzz for deterministic, non-LLM verification.*
- *The scoring asymmetry (+1 correct, +0.5 honest null, -3 hallucination) has to be designed in from day one, through weakest-link scoring and honest-null shortcuts, not bolted on later.*
- *Raw web text was messier than expected, even before AI entered the picture.*
- *LangGraph gave us hands-on experience building a real agentic pipeline.*
- *We also got to experiment meaningfully with UI/UX design.*



HONEST LIMITATIONS

Ran a fully local LLM pipeline first, but end-to-end latency was too high, so we switched to cloud APIs for the demo.

Free-tier cloud APIs came with tight context windows and request limits, which shaped how we batched and budgeted extraction calls.

Free-tier hosting for backend, frontend, and database capped our storage and instance capacity, limiting scale.

IF WE HAD MORE TIME...

OCR for image-based PDFs – Extract and verify facts from scanned brochures and infographics
Multilingual support for non-English loyalty programs using multilingual retrieval and verification.
Expanded benchmarking beyond 5 competitors with category-wide trends and what-if comparisons.

- **Shashank U** → Backend APIs, database & data pipeline.
- **Hrishikesh R Prasad** → LLM integration, LangGraph agents & verification pipeline.